



# ISLAMIC UNIVERSITY OF TECHNOLOGY

## Lab Report 8

### Livestock Management System

#### CSE 4408: System Analysis and Design Lab

##### *Lab 8: Using Data Flow Diagrams*

| Team Member                       | Student ID |
|-----------------------------------|------------|
| Ahmed Sanjeed Sadat Kibria Sadnan | 230041119  |
| Tawfiq Ahmed Tasin                | 230041129  |
| Md Shahamat Irtisham              | 230041131  |
| Syed Abtahi Bin Mashak            | 230041135  |
| Md Arkam Hossain                  | 230041157  |

***Team: Go Paloks***

***Date: July 19, 2026***

## Table of Contents

| Section | Title                                  | Page |
|---------|----------------------------------------|------|
| 1       | Introduction and System Overview       | 3    |
| 2       | Initial Logical DFDs                   | 4    |
| 3       | DFD Refinement and Error Checking      | 9    |
| 4       | Partial Physical DFD and CRUD Analysis | 11   |
| 5       | Physical DFD Partitioning              | 14   |
| 6       | Conclusion                             | 16   |
| 7       | References                             | 17   |

# 1. Introduction and System Overview

## 1.1 Organizational Context

The Livestock Management System is a proposed integrated information system for a smart farm covering poultry, fisheries and cattle operations. The present operation depends heavily on manual and disconnected records. The proposed system centralizes daily work logs, feed and inventory information, livestock health records, sales orders and management reports so that farm activities can be monitored consistently and decisions can be made from current information.

## 1.2 Purpose and Scope of the Report

This report documents the Data Flow Diagram work completed for Lab 8. It presents the initial logical DFD set, defines all major DFD components, checks balancing and naming consistency, translates the Daily Operations process into a partial physical DFD, verifies data access through a CRUD matrix, and partitions the physical fragment into implementation units. The report follows a top-down approach: Context Diagram, Diagram 0, child diagram, physical refinement and implementation partitioning.

## 1.3 DFD Principles Applied

- The context diagram contains one process, Process 0, all external entities and all major boundary data flows; it contains no data store.
- Diagram 0 decomposes Process 0 into five major business processes and introduces the persistent logical data stores required by those processes.
- The logical child diagram decomposes Process 2 and preserves the parent process inputs and outputs through vertical balancing.
- Logical DFDs describe what the business does, whereas the partial physical DFD introduces implementation details such as the mobile application, API, database tables, transaction processing and temporary validation queue.
- Data flows are named with meaningful noun phrases, processes are named with verb-based phrases, and data stores are named after the persistent data they contain.

## 2. Initial Logical DFDs

The initial logical model represents the business view of the Livestock Management System without committing to a specific programming language, database product or hardware platform. The model identifies external actors, system-boundary flows, major processes and persistent data stores before implementation details are introduced.

### 2.1 Component Definitions

#### 2.1.1 Key External Entities

| External Entity | Definition                                                                                           |
|-----------------|------------------------------------------------------------------------------------------------------|
| Farm Owner      | Requests consolidated reports and receives yield and management information for decision-making.     |
| Farm Worker     | Records daily feed usage and receives the tasks or schedules required for farm operations.           |
| Veterinarian    | Submits examinations, diagnoses, treatment and vaccination information and retrieves health history. |
| Supplier        | Provides feed or resource stock updates and receives approved purchase orders.                       |
| Buyer           | Submits product orders and receives current product availability information.                        |

*Table 1: Key external entities interacting with the system.*

#### 2.1.2 Major Boundary Data Flows

| External Entity | Data Flow                | Direction | Concise Content                                                            |
|-----------------|--------------------------|-----------|----------------------------------------------------------------------------|
| Farm Owner      | Report Requests          | Input     | Selected report type, period and filters.                                  |
| Farm Owner      | Integrated Yield Reports | Output    | Consolidated production, inventory, health, sales and operational results. |
| Farm Worker     | Daily Feed Logs          | Input     | Batch-wise feed usage and operational details.                             |
| Farm Worker     | Daily Task Schedules     | Output    | Assigned tasks, priorities and pending work.                               |
| Veterinarian    | Health Evaluation        | Input     | Examination, diagnosis, vaccination and treatment details.                 |
| Veterinarian    | Health History           | Output    | Chronological health and treatment information.                            |
| Supplier        | Stock Updates            | Input     | Feed availability, delivery and price information.                         |
| Supplier        | Purchase Orders          | Output    | Approved requests for replenishment.                                       |
| Buyer           | Product Orders           | Input     | Requested products and quantities.                                         |
| Buyer           | Product Availability     | Output    | Current sellable quantity and indicative availability.                     |

*Table 2: Major data flows crossing the system boundary.*

### 2.1.3 Core Business Process

| Core Process                    | One-Sentence Description                                                                                                                                                               |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 - Livestock Management System | Coordinates farm structure, daily operations, inventory and feed, livestock health, sales and reporting so that all livestock subsystems operate from a synchronized information base. |

*Table 3: Overarching business process represented on the context diagram.*

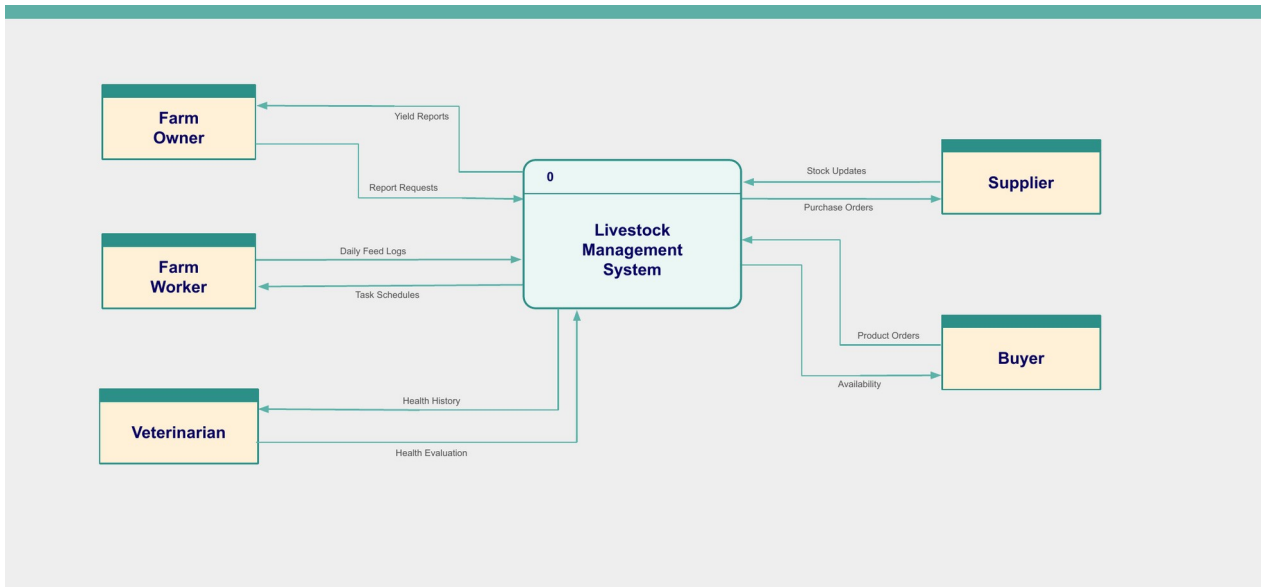
### 2.1.4 Logical Data Stores

| ID | Data Store        | Definition                                                                                                   |
|----|-------------------|--------------------------------------------------------------------------------------------------------------|
| D1 | Farm Structure    | Persistent master data for farm units, livestock categories, batches and their organizational relationships. |
| D2 | Operation Logs    | Daily feed usage, task completion and other operational transaction records.                                 |
| D3 | Inventory Records | Feed and resource stock balances, usage, replenishment and reorder information.                              |
| D4 | Health Records    | Veterinary evaluations, diagnoses, vaccinations, treatments and follow-up history.                           |
| D5 | Sales Orders      | Buyer orders, order lines, status, quantity and sales-related information.                                   |

*Table 4: Persistent logical data stores required by the system.*

## 2.2 Context Diagram

The context diagram establishes the system boundary. Process 0 represents the entire Livestock Management System, and the five external entities exchange ten major boundary flows with that process. No data store is shown at this level because the context diagram focuses only on the system scope and external interfaces.



*Figure 1: Initial context diagram of the Livestock Management System, reproduced from the team presentation.*

For consistency in the written report, the output to the owner is called Integrated Yield Reports, the worker input is called Daily Feed Logs, and the buyer output is called Product Availability. These standardized names are used in all component tables and refinement checks.

## 2.3 Logical Diagram 0

Diagram 0 decomposes the system into five major business processes and introduces the five logical data stores. Each external input and output shown on the context diagram is preserved at the system boundary. Internal flows connect reporting, daily operations, inventory, health and sales activities through shared persistent information.

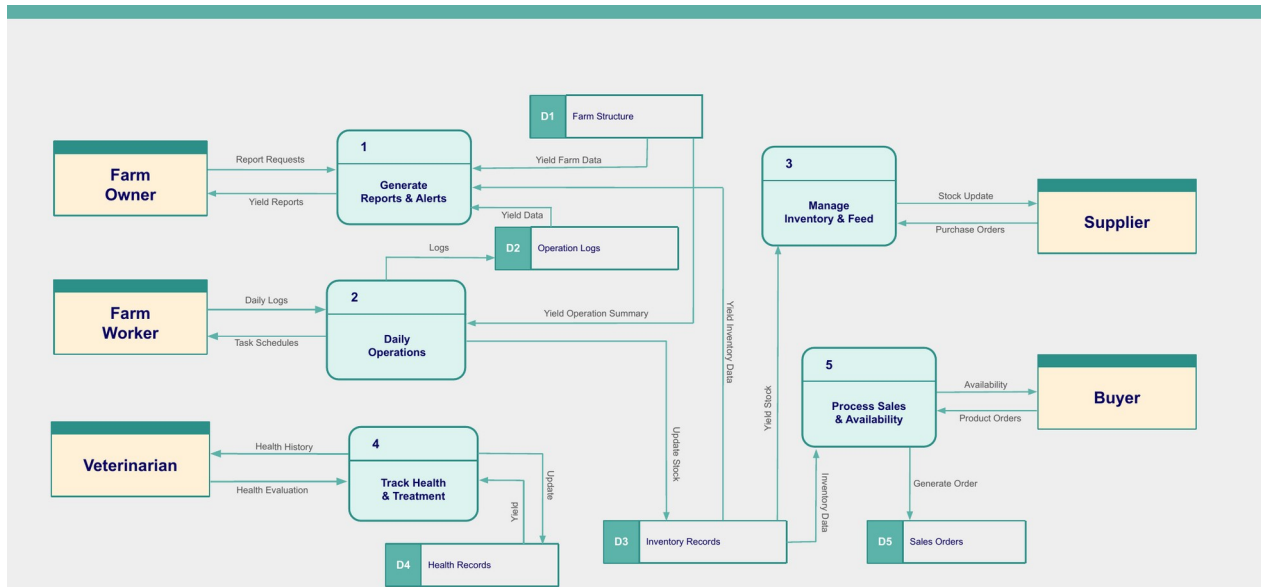


Figure 2: Initial Logical Diagram 0 showing major processes, boundary flows and logical data stores.

| Process | Name                         | One-Sentence Description                                                                                           |
|---------|------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1       | Generate Reports & Alerts    | Accepts owner report requests, reads summarized data and produces integrated reports and alerts.                   |
| 2       | Record Daily Operations      | Captures daily feed logs, maintains operation records, updates feed consumption and returns worker task schedules. |
| 3       | Manage Inventory & Feed      | Maintains feed and resource stock, processes supplier updates and generates purchase orders.                       |
| 4       | Track Health & Treatment     | Stores veterinary evaluations and provides livestock health and treatment history.                                 |
| 5       | Process Sales & Availability | Checks sellable stock, records buyer orders and returns product availability.                                      |

Table 5: Major logical processes in Diagram 0.

## 2.4 Logical Child Diagram for Process 2

Process 2 is decomposed into four subprocesses: capture the daily feed log, validate the livestock batch and quantity, generate the task schedule, and update the daily operation log. The child diagram uses D1, D2 and D3 because those stores provide farm structure, operation history and inventory information required by the parent process.

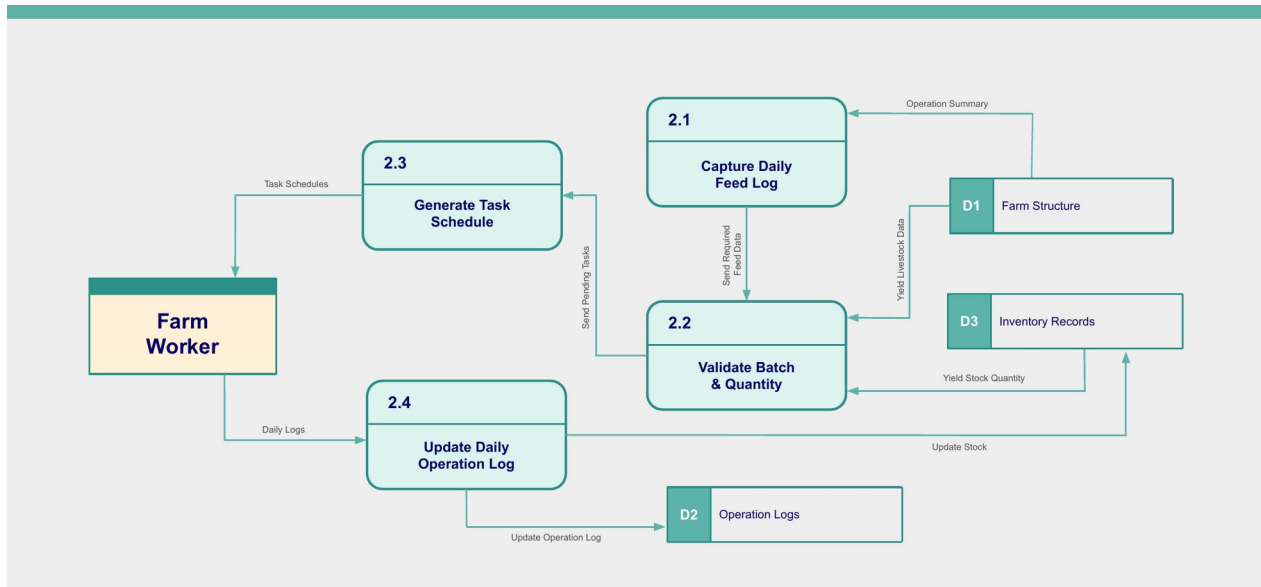


Figure 3: Initial logical child diagram for Process 2 - Daily Operations.

### 2.4.1 Vertical Balancing of Process 2

The balanced parent-child interface is interpreted as follows: Daily Feed Logs enter from the Farm Worker; Daily Task Schedules leave for the Farm Worker; D1 provides farm and batch data; D3 provides stock quantity and receives the approved feed-consumption update; D2 receives the operation-log update; and an Operation Summary leaves the child diagram for Process 1. Internal flows such as Feed Usage Entry and Validated Feed Log are introduced only inside the child diagram.



### 3. DFD Refinement and Error Checking

The initial diagrams were checked against the standard transformation, connectivity, naming and decomposition rules. Refinement is necessary because labels that are understandable in a presentation may still be inconsistent across DFD levels. The following corrections preserve the business meaning while improving formal DFD quality.

| Check Area                      | Finding                                                                                                    | Correction / Result                                                                                                                             |
|---------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Boundary-flow consistency       | The context and Diagram 0 used shortened labels such as Yield Reports, Daily Logs and Availability.        | Standardize the names as Integrated Yield Reports, Daily Feed Logs and Product Availability at every DFD level.                                 |
| Process input/output            | Every process must transform at least one incoming flow into at least one outgoing flow.                   | Confirmed for Processes 1-5 and subprocesses 2.1-2.4; the Operation Summary output is explicitly associated with Process 1.                     |
| Data-store connectivity         | A logical store cannot communicate directly with an entity or another store.                               | All D1-D5 interactions pass through a process; temporary T1 is reserved for the physical DFD only.                                              |
| Vertical balancing              | The child diagram must preserve the inputs and outputs of parent Process 2.                                | Clarified the parent interfaces: Daily Feed Logs, Daily Task Schedules, D1 lookup, D2 update, D3 read/update and Operation Summary.             |
| Meaningful process names        | Daily Operations is descriptive but does not clearly state an action.                                      | Use Record Daily Operations in formal descriptions and the CRUD matrix while retaining the presentation label in the reproduced initial figure. |
| Meaningful data-flow names      | Generic labels such as Logs, Data or Update are ambiguous.                                                 | Use specific noun phrases: Operation Log Update, Inventory Summary, Feed Consumption Update and Health Evaluation.                              |
| Persistent vs. temporary stores | A validation queue is transient and should not appear as a logical data store.                             | Show tmp_validation_queue only on the partial physical DFD and identify it as temporary store T1.                                               |
| Ownership of D1                 | Core operational processes mainly read Farm Structure, but the CRUD matrix requires maintenance ownership. | Treat Manage Farm Setup as a supporting administrative maintenance function responsible for CRUD on D1.                                         |
| Decomposition complexity        | A diagram should remain readable and normally contain no more than nine processes.                         | Diagram 0 contains five major processes and the child contains four subprocesses, both within the recommended limit.                            |

*Table 6: Error-checking findings and DFD refinement decisions.*

#### 3.1 Consistency Verification

- All ten external boundary flows in the context diagram have a matching source or destination on Diagram 0.
- Each of D1-D5 is connected to at least one process and is not connected directly to an external entity.
- Every major process and every subprocess has at least one input and one output.
- The child diagram adds only internal flows and detailed subprocesses; it does not introduce a new external entity.

- Physical controls such as validation status, rejected entries and transaction commits are not mixed into the logical DFDs.
- The CRUD matrix provides an additional check that each persistent store has appropriate create, read and update ownership.

## 4. Partial Physical DFD and CRUD Analysis

### 4.1 Partial Physical DFD Fragment

The partial physical DFD translates the Daily Operations child process into implementation components. It shows how a farm worker submits a feed log through a mobile application, how an API validates the batch and stock, how a daily task list is generated, and how an atomic database transaction updates the inventory and operation log. The fragment introduces actual data representations and physical stores while preserving the underlying logical purpose.

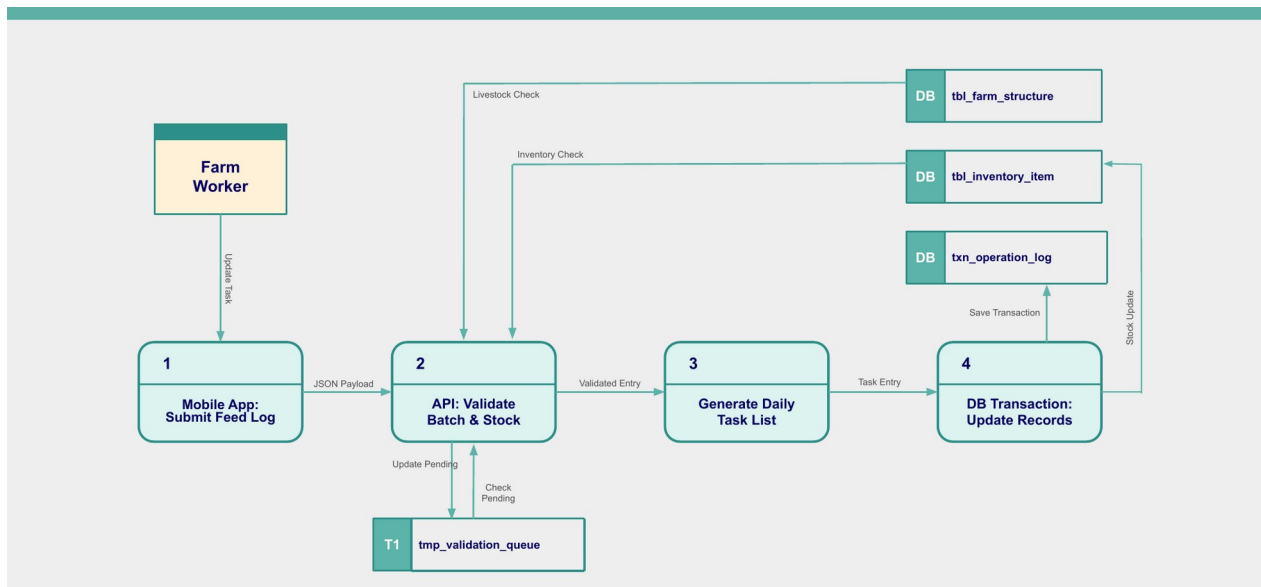


Figure 4: Partial physical DFD fragment for daily feed logging, validation, task generation and database update.

### 4.2 Physical Controls and Naming Conventions

| Physical Component             | Type                          | Control or Naming Convention                                                                                                       |
|--------------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Mobile App: Submit Feed Log    | User-interface process        | Required-field checking, authenticated worker session, client timestamp and clear confirmation/error feedback.                     |
| JSON Payload                   | Physical data flow            | Uses named fields for worker, farm unit, batch, feed item, quantity and submission timestamp.                                      |
| API: Validate Batch & Stock    | Application-service process   | Checks active batch, authorized farm unit, valid feed item, positive quantity and quantity not exceeding available stock.          |
| tmp_validation_queue           | Temporary physical data store | Keeps rejected or pending entries for correction/retry; not treated as a persistent logical store.                                 |
| Generate Daily Task List       | Application-service process   | Produces the current worker task entry only after validation succeeds.                                                             |
| DB Transaction: Update Records | Database transaction process  | Uses atomic commit/rollback, prevents partial updates, records audit timestamps and keeps inventory and operation logs consistent. |

|                    |                             |                                                                                                       |
|--------------------|-----------------------------|-------------------------------------------------------------------------------------------------------|
| tbl_farm_structure | Physical master table       | The tbl_ prefix denotes a persistent database table containing farm and batch master data.            |
| tbl_inventory_item | Physical master/stock table | Stores current quantities and is read during validation and updated within the transaction.           |
| txn_operation_log  | Physical transaction table  | The txn_ prefix identifies append-oriented operational transactions retained for audit and reporting. |

*Table 7: Physical DFD controls and naming conventions.*

### 4.3 CRUD Matrix

The CRUD matrix checks which business process creates, reads, updates or deletes records in each logical store. It confirms that reporting is read-only, operational processes have the minimum required access, and transactional records are not routinely deleted because they support traceability and auditability.

| Process                      | D1 Farm Structure | D2 Operation Logs | D3 Inventory Records | D4 Health Records | D5 Sales Orders |
|------------------------------|-------------------|-------------------|----------------------|-------------------|-----------------|
| Manage Farm Setup            | C/R/U/D           | -                 | -                    | -                 | -               |
| Record Daily Operations      | R                 | C/R/U             | U                    | R                 | -               |
| Manage Inventory & Feed      | R                 | R                 | C/R/U                | R                 | -               |
| Track Health & Treatment     | R                 | R                 | -                    | C/R/U             | -               |
| Process Sales & Availability | R                 | -                 | R/U                  | -                 | C/R/U           |
| Generate Reports & Alerts    | R                 | R                 | R                    | R                 | R               |

*Table 8: CRUD matrix for the major processes and logical data stores.*

C = Create, R = Read, U = Update and D = Delete. Delete access is limited to controlled maintenance of farm-structure master data. Operation, health, inventory and sales transactions should normally be retained and corrected through status changes or compensating transactions rather than deletion.

## 5. Physical DFD Partitioning

Partitioning groups physical DFD processes into implementation units. The daily-operations fragment is divided into a User Interface partition, an Application Services partition and a Database Layer partition. Dashed boundaries show which processes and stores are implemented together and where controlled interfaces are required.

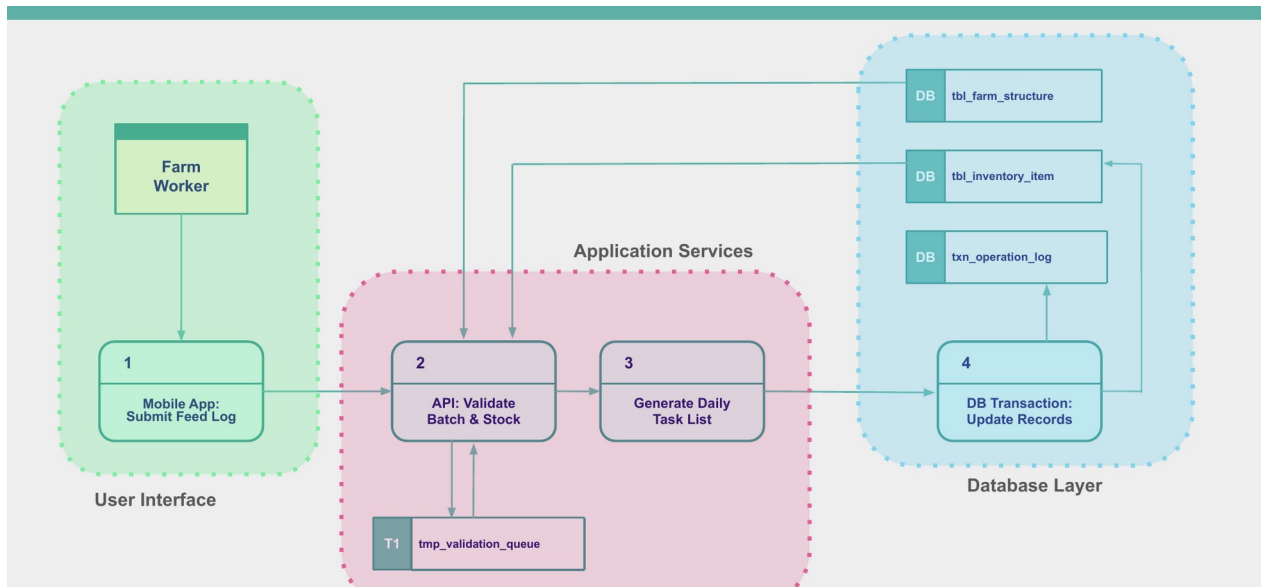


Figure 5: Partitioned physical DFD fragment showing the User Interface, Application Services and Database Layer.

### 5.1 Partitioning Rationale and Requirement Linkage

| Partition            | Contained Components                                           | Rationale                                                                                                                              | System Requirements Supported                                                                    |
|----------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| User Interface       | Farm Worker and Mobile App: Submit Feed Log                    | Separates worker interaction from business and database logic; provides a simple mobile-friendly form and displays task notifications. | Usability, adoption by farm workers, reduced entry error and faster daily logging.               |
| Application Services | API validation, task generation and temporary validation queue | Centralizes reusable business rules, prevents direct client access to data stores, and supports correction/retry of invalid entries.   | Consistent validation, reduced wastage, prompt alerts, maintainability and scalable processing.  |
| Database Layer       | DB transaction plus farm, inventory and operation tables       | Isolates persistent master and transaction data, enforces constraints, atomicity, role-based access and audit records.                 | Data integrity, security, accountability, reliable inventory depletion and historical reporting. |

Table 9: Partitioning rationale tied to the Livestock Management System requirements.

### 5.2 Justification of the Partition Boundaries

- Different user groups: farm workers interact with the mobile interface, whereas application

and database components are managed by technical or administrative roles.

- Security: the farm worker interface cannot access database tables directly; requests pass through the API, where role and data validation controls are applied.
- Similar tasks: validation and task generation are grouped as application services because both use business rules and coordinated data lookups.
- Efficiency and consistency: inventory and operation updates are grouped in one database transaction so that both records reflect the same validated event.
- Timing: invalid submissions can remain in a temporary validation queue and be corrected without blocking valid daily operations.
- Maintainability: the interface, rules and data persistence can be modified or scaled independently while their contracts remain stable.

### **5.3 Partition Interface Rules**

The User Interface sends structured requests to Application Services and receives task or error responses. Application Services may read the required master and stock data but must request all changes through a controlled database transaction. The Database Layer never initiates a direct flow to the Farm Worker. This separation supports role-based access control, testability and clear responsibility for validation and persistence.

## 6. Conclusion

The completed DFD report presents a coherent top-down model of the Livestock Management System. The context diagram defines the system boundary and external exchanges; Diagram 0 identifies the five major business processes and persistent stores; and the child diagram explains the internal logic of Daily Operations. Error checking standardizes names, confirms process and store connectivity, and preserves vertical balance. The partial physical DFD then introduces the mobile application, API validation, temporary queue, physical tables and atomic transaction. Finally, the CRUD matrix and three-layer partitioning verify data ownership, security, integrity, maintainability and alignment with the farm system requirements.